01. THÔNG TIN NHÓM VÀ MÔ HÌNH

BÁO CÁO GIỮA KỲ • NHÓM 10 • MINIMAX VÀ ALPHA–BETA

Học phần: Biểu diễn tri thức và tìm kiếm nâng cao

Giảng viên: TS. Nguyễn Bích Vân — IAI–UET–VNU

Nhóm / lớp: Nhóm 10 - UET.AI2014_1 Ngày giao 16/10/2026 / ngày nộp: [Điền]

Python: 3.14.4 / hệ điều hành: Linux (WSL2, x86_64) / máy dùng chung: Máy cá nhân của nhóm trưởng.

Hướng dẫn: thay [Điền] bằng bài làm và kết quả thật; thêm dòng hoặc giãn ô khi cần. Mẫu trống 6 trang; báo cáo hoàn thiện 6–8 trang nội dung chính, không tính bài tính tay. Chọn Word hoặc LaTeX, không phải điền cả hai.

Thành viên / MSSV	**Chủ trì**	**Test phụ trách**	**Kiểm tra chéo**
1. Hoàng Trung Hiếu / 25022849	Mô hình cây, Minimax; bài B1	Test 1: điểm âm, bảo toàn input; test 2: utility 0, đường đi tối ưu	Đọc và đối chiếu vết Alpha–Beta trên T0
2. Trần Nguyên Hiển Minh / 25022950	Alpha–Beta, cắt tỉa; bài B2 và B4	Test 3: T1, value/action; test 4: cận và số đếm T0	Đối chiếu bảng Minimax của Hiếu với `returns`
3. Lê Bảo Ngân / 25022958	Dữ liệu N1/N2, tham số và kiểm thử	Test 5: tham số T0(t); test 6: N2 gốc MIN; test 7: dữ liệu lá không hợp lệ	Tái tạo kết quả của Hiếu và Minh; đối chiếu value/action
4. Nguyễn Hoàng Anh / 25022772	Thực nghiệm 24+8 cấu hình, tích hợp, báo cáo và slide	Test 8: hòa thật theo LR/RL; test 9: bật/tắt trace và tích hợp JSON	Kiểm tra JSON/CSV, đủ 9 test và các lệnh chạy

1. Bài toán nhóm đang giải

Trạng thái, hành động, chuyển trạng thái, lượt đi và kết thúc. Mỗi nút của cây T_0 là một trạng thái: gốc R là trạng thái ban đầu; 14 nút trong còn lại ($A, B, A1, A2, B1, B2, A11, \ldots, B22$) là trạng thái trung gian; 16 lá $L01, \ldots, L16$ là trạng thái kết thúc. Hành động tại một nút trong là chọn một trong hai con trực tiếp (trái hoặc phải). Hàm chuyển Result(s, a) trả về nút con tương ứng, ví dụ Result$(A, \text{phải}) = A2$. Lượt đi luân phiên theo tầng: MAX đi ở R và tầng $A1, A2, B1, B2$; MIN đi ở tầng A, B và tầng $A11, \ldots, B22$. Trò chơi kết thúc khi chạm lá L_x ($x \in [01, 16]$); thứ tự LR/RL chỉ đổi thứ tự xét con, không đổi tên hay vị trí lá.

Utility có ý nghĩa gì? Vì sao không cộng các điểm trên đường? Utility là lợi ích cuối cùng của ván chơi nhìn từ phía MAX, chỉ gắn với trạng thái kết thúc (lá): MAX muốn tăng, MIN muốn giảm cùng một con số. Các nút trong không có điểm riêng; giá trị của chúng là giá trị Minimax suy ra từ lá. Vì vậy cộng điểm dọc đường là đếm lặp và biến bài toán đối kháng thành bài toán tìm đường có tổng lớn nhất, bỏ qua việc MIN chọn ngược lại. Utility 0 là một kết quả hợp lệ (ví dụ hòa), khác hoàn toàn với `None` ("không phải lá").

Tại sao chọn lá lớn nhất chưa phải chiến lược Minimax? Một đường chơi khác chính sách thế nào? Lá lớn nhất $L14 = 101$ nằm dưới $B \to B2 \to B21$. Nếu MAX chọn B thì quyền quyết định tiếp theo thuộc về MIN; MIN sẽ chọn $B1$ (giá trị 4) chứ không chọn $B2$ (giá trị 100). Do đó chọn B chỉ đảm bảo 4, kém hơn 10 khi chọn A; Minimax giả định đối thủ chơi tối ưu nên không thể hy vọng MIN đi vào $B2$. Một đường chơi (ví dụ $R \to A \to A1 \to A11 \to L01$) chỉ là một chuỗi nước đi định sẵn, ngầm giả định đối thủ đi đúng như ta đoán. Chính sách thì quy định nước đi của MAX tại *mọi* nút MAX có thể gặp, ứng với mọi phản ứng của MIN: chọn A; nếu MIN chọn $A1$ thì chọn $A12$, nếu MIN chọn $A2$ thì chọn $A22$. Giá trị 10 là mức chính sách này đảm bảo dù MIN đi thế nào; đường $R \to A \to A1 \to A12 \to L04$ chỉ là đường xảy ra khi cả hai bên cùng chơi tối ưu.

Phần mã dùng nguyên, phần nhóm sửa/thêm. Dùng nguyên, không sửa thuật toán: `tree_model.py` (`make_tree`, `walk`, `validate_tree`, `load_cases`), `search.py` (`solve`), `data_cases.py`, `run.py`, `tao_du_lieu.py`, `chay_kiem_thu.py`, `dong_goi.py`. Nhóm thêm: `test_nhom.py` (9 test, phân bổ 2–2–3–2), `du_lieu_nhom.json` (N1, N2 sinh bằng `tao_du_lieu.py`), thư mục `output/` (kết quả chạy thật).

02. TRÌNH BÀY THUẬT TOÁN

2. Minimax và Alpha–Beta bằng lời của nhóm

Công thức truy hồi Minimax (lá / MAX / MIN).

$$V(u) = \begin{cases} \text{utility}(u) & \text{nếu } u \text{ là lá,} \\ \max_{c \in \text{con}(u)} V(c) & \text{nếu } u \text{ là nút MAX,} \\ \min_{c \in \text{con}(u)} V(c) & \text{nếu } u \text{ là nút MIN.} \end{cases}$$

Hành động tại u là con c đầu tiên (theo thứ tự LR hoặc RL) đạt giá trị trên; khi hòa giữ con xét trước nhờ so sánh nghiêm. Mã giả của nhóm:

```
def minimax(u):
    if u is leaf:                       # kiem tra bang "is not None", 0 van la la
        return utility(u), None
    if u is MAX: best = -inf
    else:        best = +inf
    action = None
    for c in children(u) in order LR or RL:
        value, _ = minimax(c)
        if (u is MAX and value > best) or (u is MIN and value < best):
            best, action = value, c     # so sanh nghiem: hoa giu con truoc
    return best, action
```

Vì sao khởi tạo $-\infty$ / $+\infty$? Vì $\max(-\infty, x) = x$ và $\min(+\infty, x) = x$ với mọi utility hợp lệ x: con đầu tiên luôn ghi đè `best`, nên giá trị được chọn lấy từ kết quả của một con đã xét. Nếu khởi tạo `best` tại MAX bằng 0 trên cây toàn điểm âm (T2, test 1), `best` có thể giữ nguyên 0 thay vì một kết quả âm, còn `action` vẫn là `None`, vì không có kết quả con nào lớn hơn 0.

Vì sao utility 0 không bị coi là "chưa kết thúc"? Lá được nhận biết bằng `node.utility is not None`, không phải `if node.utility`. Nếu dùng phép thử đúng/sai, lá có utility 0 bị xem như nút trong không có con và kết quả sai; `None` mới là dấu hiệu "không phải lá" (test 2 kiểm tra điều này).

Giải thích alpha, beta và fail-soft:
Trên đường tìm kiếm hiện tại, MAX đã có phương án để nhận được ít nhất α điểm, còn MIN đã có phương án để MAX nhận không quá β điểm. Cả hai ngưỡng đều tính theo utility của MAX và được truyền từ các nút tổ tiên xuống. Gốc nhận $[-\infty, +\infty]$; mỗi con nhận cửa sổ hiện có của cha, không khởi tạo lại giữa các con cùng cha. Ví dụ trên T_0, sau khi $A1$ trả về 10, nút A (MIN) đã có lựa chọn $A1$ để giới hạn điểm của MAX ở mức 10. Vì thế, A đặt $\beta = 10$ và truyền ngưỡng này khi xét $A2$. Ngưỡng đó không khẳng định giá trị của $A2$ phải nhỏ hơn hoặc bằng 10; nó cho biết MIN đã có một lựa chọn khác đạt mức 10.

Sau khi nhận kết quả từ con, cập nhật `best` và `action` bằng so sánh nghiêm, rồi kiểm tra dừng: MAX khi $best \geq \beta$, MIN khi $best \leq \alpha$. Khi đó, các con còn lại không thể cải thiện lựa chọn của tổ tiên nên được bỏ qua. Nếu chưa dừng, MAX cập nhật $\alpha = \max(\alpha, best)$, MIN cập nhật $\beta = \min(\beta, best)$ để xét con tiếp theo. Fail-soft trả nguyên `best`, kể cả khi nằm ngoài cửa sổ, thay vì ép về α hoặc β; vì vậy số trả có thể chỉ là cận, không phải giá trị Minimax chính xác của nút. Với cùng cây, thứ tự duyệt và quy tắc xử lý hòa, Alpha–Beta bảo toàn cả `value` và `action` tại gốc như Minimax.

Khối xử lý	**Mã giả / giải thích của nhóm**	**Tệp và dòng**
Kiểm tra lá	Nếu u là lá: trả `(utility, None)`. So sánh với `None` nên lá 0 vẫn là lá.	`search.py` dòng 32–36
Khởi tạo best/action	MAX: $best = -\infty$; MIN: $best = +\infty$; `action = None`.	`search.py` dòng 38–40
Gọi đệ quy	Duyệt con theo LR hoặc đảo ngược nếu RL; `score = visit(child, ...)` tương đương `minimax(c)`.	`search.py` dòng 41–43
Cập nhật hành động	MAX: nếu $score > best$; MIN: nếu $score < best$ thì `best, action = score, c`. So sánh nghiêm: hòa giữ con xét trước.	`search.py` dòng 44–45
Truyền / cập nhật cửa sổ	Gọi con với α, β hiện có. Nếu chưa dừng: MAX cập nhật $\alpha = \max(\alpha, best)$; MIN cập nhật $\beta = \min(\beta, best)$.	`search.py` dòng 43, 59–60; gốc dòng 71
Điều kiện cắt / trả về	MAX dừng khi $best \geq \beta$; MIN dừng khi $best \leq \alpha$. Chỉ đếm cắt khi còn con bị bỏ; trả `best, action` và gán cờ theo cửa sổ lúc vào.	`search.py` dòng 49–69

3. Những quy tắc không được nhầm

Tại MAX và MIN, số trả sớm là cận theo hướng nào?
MAX cắt khi đạt ngưỡng β: số trả r là cận dưới (`LOWER`), nghĩa là $V(u) \geq r$. MIN cắt khi đạt ngưỡng α: r là cận trên (`UPPER`), nghĩa là $V(u) \leq r$. Mã phân loại bảo thủ theo cửa sổ lúc vào $(\alpha_{in}, \beta_{in})$: $r \leq \alpha_{in}$ là `UPPER`, $r \geq \beta_{in}$ là `LOWER`, còn lại là `EXACT`; lá luôn `EXACT`. Cờ cận không khẳng định nút đã cắt, vì ngay cả khi đã xét hết con, mã vẫn phân loại theo cửa sổ này.

Vì sao so sánh chọn action nghiêm nhưng cắt dùng cả dấu bằng? MAX chỉ đổi action khi *score* > *best*, MIN khi *score* < *best*, để giữ con xét trước lúc hòa. Một cận bằng best chưa chứng minh hai nhánh có cùng giá trị thật (T1 là ví dụ). Cắt dùng cả dấu bằng vì nhánh còn lại không thể cải thiện giá trị tổ tiên đã có; việc bỏ nó không đổi giá trị gốc và phù hợp quy tắc giữ con trước.

Trường	Nhóm hiểu và kiểm tra thế nào?
visited = expanded + leaves	Mỗi lần vào `visit` đếm một nút: hoặc là lá đã đọc, hoặc là nút trong được mở rộng. T0 (Alpha–Beta, LR): $22 = 12 + 10$; test 4 kiểm tra cả từng số và đẳng thức.
cutoffs / pruned_branches	`cutoffs` đếm lần dừng có con còn lại; `pruned_branches` đếm cạnh tới các con trực tiếp bị bỏ, không đếm toàn bộ lá bên dưới. T0/LR có 3 lần cắt, 3 cạnh bỏ nhưng 6 lá không đọc. Vượt ngưỡng ở con cuối không tăng hai bộ đếm.
unvisited_nodes / leaf_order	`unvisited_nodes` liệt kê mọi nút không được gọi `visit`, gồm cả nút trong; `leaf_order` là thứ tự thực sự đọc lá. T0 (Alpha–Beta, LR) có 9 nút không thăm và đọc 10 lá; test 4 đối chiếu tập nút bỏ và đúng thứ tự lá.
principal_variation / returns	`principal_variation` chỉ được dựng từ Minimax đầy đủ; Alpha–Beta trả `None` cho trường này. `returns` lưu value, action và cờ cận của từng nút đã thăm; không coi các action nội bộ của Alpha–Beta là đường chơi tối ưu đã được chứng minh.

Thời gian và bộ nhớ. Với hệ số phân nhánh b và độ sâu d, Minimax có thời gian xấu nhất $O(b^d)$ do phải duyệt toàn bộ cây. Alpha–Beta cũng có thời gian xấu nhất $O(b^d)$ khi không cắt được nhánh nào.

Về bộ nhớ, tại mỗi thời điểm, các lời gọi đệ quy đang hoạt động nằm trên một đường từ gốc xuống nút hiện tại, nên stack đệ quy tốn $O(d)$. Cây được dựng sẵn trong bộ nhớ tốn $O(N)$, với N là tổng số nút.

Trong hàm `solve()` của tệp `search.py`, chương trình còn lưu các nút đã thăm (`visited_nodes`), thứ tự đọc lá (`leaf_order`) và kết quả trả về tại từng nút (`returns`). Chương trình cũng tạo danh sách toàn bộ nút để xác định các nút không được thăm (`unvisited_nodes`). Các cấu trúc này cần bộ nhớ phụ $O(N)$ trong trường hợp xấu nhất, kể cả khi tắt trace. Khi bật trace, chương trình lưu thêm $O(N)$ sự kiện trên cây nhị phân của bài, nên tổng bộ nhớ phụ vẫn là $O(N)$.

03. ĐỐI CHIẾU TÍNH TAY VÀ MÃ

PDF bài tính tay: `TinhTay.pdf`, trang 1–14. Ngày ghi dự đoán trước khi chạy: 22/10/2026.

4. T0: Minimax và Alpha–Beta

Minh chứng tính tay: `TinhTay.pdf`, trang 1–3 (B1) và trang 4–6 (B2). Đối chiếu với `output/bat_buoc.json`, các bản ghi có `case=T0`, `order=LR` cho hai thuật toán.

Nội dung	Dự đoán bằng tay	Kết quả code / có khớp?
Giá trị / hành động gốc	$A = \min(10, 13) = 10$; $B = \min(4, 100) = 4$; $R = 10$, chọn A (LR).	Minimax và Alpha–Beta đều trả $(10, A)$; khớp.
Đường chơi tối ưu từ Minimax	$R \to A \to A1 \to A12 \to L04$, utility 10.	`principal_variation` của Minimax đúng chuỗi dự đoán; Alpha–Beta không xuất PV.
Visited / expanded / leaves	Minimax: 31/15/16. Alpha–Beta: bỏ 9 nút (3 nút trong, 6 lá), còn 22/12/10.	Minimax: 31/15/16; Alpha–Beta: 22/12/10; khớp.
Gốc nhánh cắt / số cạnh	Tại $B11$ bỏ $L10$; tại $B12$ bỏ $L12$; tại B bỏ $B2$: 3 cạnh.	`pruned_roots` là $[L10, L12, B2]$; `cutoffs` và `pruned_branches` đều bằng 3; khớp.
Lá không đọc / mọi nút không thăm	6 lá: $L10, L12, L13, L14, L15, L16$. Thêm $B2, B21, B22$ thành 9 nút không thăm.	Alpha–Beta bỏ đúng các nút trên, đọc $L01$–$L09$ rồi $L11$; Minimax đọc đủ 16 lá. Khớp.

Trích một lần cắt: nút $B11$ (MIN), alpha/beta $[10, +\infty]$ (MAX đã có 10 từ A), số trả 4 sau khi đọc $L09$, nhánh bỏ $B11 \to L10$ vì $4 \le 10$, hướng cận `UPPER`: $V(B11) \le 4$, không phải cận dưới.

Khi chuyển từ A sang B và ý nghĩa phần cây bị cắt. Sau khi xét A, gốc giữ $\alpha = 10$ khi chuyển sang B; không khởi tạo lại α giữa hai nút con. Cây con $B2$ bị cắt hoàn toàn nên lần chạy Alpha–Beta này không cung cấp giá trị thật của các nút trong $B2$. Giá trị $V(B2) = 100$ dùng trong bảng tính tay được suy ra từ tính tay hoặc Minimax đầy đủ; không lấy từ lần chạy Alpha–Beta trên.

5. T(t): thay riêng L01

Minh chứng tính tay: `TinhTay.pdf`, trang 7–10 (B3); bảng đường chơi theo khoảng ở trang 8, ba trường hợp cụ thể ở trang 9–10.

Công thức giá trị gốc theo từng khoảng t: $V(R) = 10$ khi $0 \le t < 10$; $V(R) = t$ khi $10 \le t \le 12$; $V(R) = 12$ khi $12 < t \le 20$.

Đường chơi tối ưu theo quy tắc hòa LR.

$$\begin{aligned} 0 \le t < 10: &\quad R \to A \to A1 \to A12 \to L04, \\ 10 \le t \le 12: &\quad R \to A \to A1 \to A11 \to L01, \\ 12 < t \le 20: &\quad R \to A \to A1 \to A11 \to L02. \end{aligned}$$

Tại $t = 10$, $A1$ chọn $A11$ do hai con cùng giá trị 10 và $A11$ được xét trước. Tại $t = 12$, $A11$ vẫn chọn $L01$ do hai lá cùng giá trị 12; chỉ khi $t > 12$ mới chuyển sang $L02$.

Ngưỡng xuất hiện nhánh cắt L04. Sau khi xét $A11$, $A1$ truyền $\alpha = \min(t, 12)$ xuống $A12$. Tại $A12$ (MIN), đọc $L03 = 11$ thì cắt $L04$ khi $11 \le \min(t, 12)$, tương đương $t \ge 11$. Như vậy, ngưỡng cắt $t = 11$ khác với các mốc đổi đường chơi ở 10 và 12.

t	Value / action	Đường chơi thật	Lá AB / tệp
10	10 / A	R–A–A1–A11–L01	10 / `bat_buoc.csv`
11	11 / A	R–A–A1–A11–L01	9 / `bat_buoc.csv`
14	12 / A	R–A–A1–A11–L02	9 / `bat_buoc.csv`

Tệp trong bảng là `output/bat_buoc.csv`; các trường hợp $t = 10, 11, 14$ lần lượt ứng với dòng 17, 19, 23, tính cả dòng tiêu đề. Cả Minimax và Alpha–Beta LR đều cho value/action như bảng; đường chơi được lấy từ trường `principal_variation` của Minimax trong tệp `output/bat_buoc.json`.

6. T1: kết quả điểm đúng nhưng action sai

Minh chứng tính tay: `TinhTay.pdf`, trang 11 (B4); đối chiếu các bản ghi `case=T1`, `order=LR` trong `output/bat_buoc.json`.

Giá trị thật A/B/R:
$A = \min(10, 12) = 10$, $B = \min(10, 2) = 2$, $R = \max(10, 2) = 10$, nên MAX phải chọn A. Số B trả và hướng cận: Với LR, xét A trước làm $\alpha_R = 10$. Tại B (MIN), đọc $L03 = 10$ đã đạt $best \leq \alpha$ nên cắt $L04 = 2$ và trả 10 với cờ `UPPER`: $V(B) \leq 10$, không có nghĩa $V(B) = 10$.

Ảnh hưởng của >= khi đổi action; phản ứng của MIN; test phát hiện:
Nếu đổi phép chọn action ở MAX từ $>$ thành $\geq$, gốc sẽ lấy số cận 10 của B làm hòa với giá trị chính xác 10 của A rồi đổi sang B. Kết quả báo $(10, B)$ có điểm đúng nhưng hành động sai: khi chơi thật, MIN chọn $L04 = 2$, MAX chỉ nhận 2. Giữ so sánh nghiêm làm gốc giữ A. Test 3, `test_03_T1_false_tie`, kiểm tra đồng thời cặp `(value, action) = (10, A)` cho cả Minimax và Alpha–Beta; chỉ kiểm tra value sẽ không bắt được lỗi.

Không cần bỏ dấu bằng trong điều kiện cắt; để sửa lỗi chọn hành động, cần giữ so sánh nghiêm khi cập nhật action: MAX dùng $score > best$, MIN dùng $score < best$.

04. KẾT QUẢ CHUNG VÀ DỮ LIỆU RIÊNG

JSON/CSV chung: output/bat_buoc.* (24 cấu hình).

Lệnh và ngày chạy: `python3 run.py --batch --tag bat_buoc`, ngày 23/10/2026.

Cây / thứ tự	Value / action	Lá MM	Lá AB	Tiết kiệm
T0/LR	10 / A	16	10	37.5% (bỏ 6 lá)
T0/RL	10 / A	16	16	0% (0 lá)
T1/LR	10 / A	4	3	25% (1 lá)
T1/RL	10 / A	4	4	0% (0 lá)
T2/LR	-4 / A	8	8	0% (0 lá)
T2/RL	-4 / A	8	7	12.5% (1 lá)

Một phép tính tỷ lệ tiết kiệm: Trên T0/LR, Minimax duyệt 16 lá, Alpha–Beta duyệt 10 lá, tỷ lệ tiết kiệm lá là $(16-10)/16 = 37.5\%$. Một trường hợp không tiết kiệm lá: T0/RL (hoặc T1/RL, T2/LR) duyệt trúng nhánh xấu trước nên không cắt được lá nào (tiết kiệm 0%).

Năm mốc tham số và hai dòng affine nằm ở CSV các dòng từ 14 đến 25. Dự đoán / kết quả của u ↦ 2u-7: Giá trị gốc $V(R) = 2 \times 10 - 7 = 13$, hành động giữ nguyên là A, số lá và vị trí cắt tỉa không đổi (Alpha–Beta đọc 10 lá, 3 lần cắt); khớp dự đoán lý thuyết.

7. Hai cây riêng N1/N2

Nội dung	N1 (16 lá, MAX)	N2 (8 lá, MIN)
Dãy utility (hoặc dẫn JSON)	24,22,3,-3,21,13,18,-3 -5,29,-5,21,-9,29,29,-3	14,-15,11,12 -19,-6,17,-13
Seed / cách tạo	2579 / `tao_du_lieu.py`	2579 / `tao_du_lieu.py`
Giá trị / action LR	13 / A	−13 / B
Giá trị / action RL	13 / A	−13 / B
So sánh số lá Minimax và Alpha–Beta	Minimax: 16; Alpha–Beta: 9 (LR), 16 (RL)	Minimax: 8; Alpha–Beta: 8 (LR), 6 (RL)

Kỳ vọng tính tay độc lập của N2: $V(R) = \min(\max(-15, 11), \max(-19, -13)) = -13$, hành động B; xem `TinhTay.pdf`, trang 12–14 (N2).

JSON/CSV riêng: output/tu_tao.* có 8 cấu hình; xác nhận: đủ 8 cấu hình, N2 trả −13 / B ở cả LR/RL.

Đo thời gian (tùy chọn): Không thực hiện.

05. KIỂM THỬ VÀ PHÁT HIỆN SAI LẦM

Tệp test_nhom.py; log output/ket_qua_test.txt. Lệnh: `python3 -m unittest -v test_nhom`; ngày chạy: 23/10/2026.

Tổng test: 9; đạt: 9; thất bại/lỗi: 0.

STT	Test / dữ liệu	Kỳ vọng và cách suy ra	Kết quả	Người
1	`test_01_am_va_khong_sua_input`; 8 lá âm	$A=-3$, $B=-4$, $R=-3$, chọn A ở cả Minimax và Alpha–Beta; mảng lá đầu vào giữ nguyên.	PASS	Hiếu
2	`test_02_utility_zero_va_duong_di_toi_uu`; 8 lá (có 0)	Minimax trả $(0, A)$; PV: R, A, A1, L01; đọc đủ 8 lá.	PASS	Hiếu
3	`test_03_T1_false_tie`; T1: $[10, 12, 10, 2]$, LR	$A=10$, $B=2$; cả Minimax và Alpha–Beta phải trả $(10, A)$, không chấp nhận hòa giả $(10, B)$.	PASS	Minh
4	`test_04_T0_pruning_bounds_and_counts`; T0, Alpha–Beta, LR	$(10, A)$; bỏ $L10, L12, B2$; cận `UPPER` tại $B11, B12, B$; $22 = 12 + 10$, 3 lần cắt/3 cạnh. Kiểm cả thứ tự lá và tập nút bỏ.	PASS	Minh
5	`test_05_t0_parameter`; $t = 10, 11, 14$	Minimax và Alpha–Beta trả value $10, 11, 12$, đều chọn A; Alpha–Beta đọc $10, 9, 9$ lá tương ứng.	PASS	Ngân
6	`test_06_n2_root_min`; N2 từ JSON	Gốc MIN; cả Minimax và Alpha–Beta với LR/RL trả $(-13, B)$.	PASS	Ngân
7	`test_07_invalid_leaf_data`; 3 lá, chuỗi, bool	Báo `ValueError` với 3 lá, utility chuỗi hoặc bool.	PASS	Ngân
8	`test_08_hoa_that_theo_LR_RL`; $[2, 5, 2, 7]$	Hai nhánh cùng giá trị 2; cả Minimax và Alpha–Beta chọn A khi LR, B khi RL.	PASS	Hoàng Anh
9	`test_09_trace_bat_tat_va_tich_hop_json`; N1	Bật/tắt trace Alpha–Beta giữ nguyên kết quả và số đếm được kiểm; trace tắt rỗng, bật có sự kiện; visited = expanded + leaves.	PASS	Hoàng Anh

Bốn test có kỳ vọng tính tay độc lập trong mã nguồn:

- **Test 1:** $A=-3$, $B=-4 \implies R=-3$, chọn action A.
- **Test 2:** $A=0$, $B=-1 \implies R=0$, chọn action A, PV: $R \to A \to A1 \to L01$.
- **Test 3:** $A=10$, $B=2 \implies R=10$, chọn action A (cây T_1).
- **Test 4:** Vết tỉa trên cây T_0: B nhận $\alpha=10$, cắt $L10$ tại $B11$, cắt $L12$ tại $B12$, cắt toàn bộ cây con $B2$ tại B.

Ghi chú: Các bước tính toán chi tiết được trình bày đầy đủ trong docstring của từng test.

Test T1 có kiểm cả value lẫn action không? Có. Test 3 dùng `assertEqual` cho cả cặp `(value, action)` với kỳ vọng `(10, 'A')` ở hai thuật toán; kỳ vọng được suy ra từ $A=10$, $B=2$, không lấy kết quả của thuật toán này làm đáp án cho thuật toán kia.

8. Hai lỗi nhóm đã phân tích

Mỗi lỗi được thử riêng trên bản sao của hàm `solve`, dùng lại test của nhóm; các số dòng dưới đây tham chiếu tới `search.py`.

Mục	**Lỗi 1**	**Lỗi 2**
Tệp / dòng / thay đổi gây lỗi	`search.py`, dòng 39: tại MAX, thay `-math.inf` bằng 0; MIN vẫn khởi tạo bằng `math.inf`.	`search.py`, dòng 44: tại MAX, đổi $score > best$ thành $score \geq best$; MIN vẫn dùng $<$.
Dự đoán trước khi chạy	Với cây toàn điểm âm, MAX giữ best bằng 0 và không chọn được action hợp lệ, gây sai kết quả hoặc lỗi khi dựng đường chơi.	Trên T1/LR, gốc coi cận trên 10 từ B là hòa với giá trị 10 của A rồi chọn B; khi chơi thật, MIN chọn lá có utility 2.
Dữ liệu / test phát hiện	`test_nhom.py`, test 1: lá $[-3, -7, -1, -5, -9, -2, -6, -4]$; kỳ vọng $(value, action) = (-3, A)$.	`test_nhom.py`, test 3: T1 có lá $[10, 12, 10, 2]$, LR; kỳ vọng $(value, action) = (10, A)$.
Kết quả thực tế / cách sửa	Test 1 báo `ERROR: StopIteration` khi Minimax dựng `principal_variation`, do action tại gốc là `None`. Cách sửa: khôi phục khởi tạo MAX bằng `-math.inf`.	Test 3 báo `FAIL` ở Alpha–Beta: trả $(10, B)$ thay vì $(10, A)$. Cách sửa: khôi phục so sánh nghiêm $score > best$ tại MAX.

Các trường hợp chưa kiểm hoặc lỗi còn tồn tại: Hai thử nghiệm trên chưa khảo sát lỗi reset cửa sổ ở từng con của gốc. Chưa phát hiện lỗi trong bản mã chính qua 9 test nhóm hiện có.

06. NHẬN XÉT, TÁI TẠO VÀ BẢO VỆ

9. Ba kết luận có minh chứng

Kết luận 1 — LR/RL và số lá. Thứ tự duyệt ảnh hưởng rõ đến hiệu quả cắt tỉa của Alpha–Beta. Trên T0, LR đọc 10/16 lá, tiết kiệm 37,5%, còn RL đọc đủ 16 lá. Ngược lại, trên N2, RL đọc 6/8 lá, còn LR đọc đủ 8 lá. Không có thứ tự nào luôn tốt hơn: xét nhánh có lợi sớm giúp thiết lập cận để cắt các nhánh sau. Trong những cấu hình này, đổi thứ tự vẫn giữ nguyên value và action tại gốc.

Kết luận 2 — Đổi thang utility. Phép đổi utility $u \mapsto 2u - 7$ trên T0 làm giá trị gốc tăng từ 10 thành 13, trong khi action vẫn là A. Alpha–Beta LR tiếp tục đọc 10 lá và có 3 lần cắt, giống cấu hình ban đầu. Phép biến đổi có hệ số dương bảo toàn thứ tự và quan hệ bằng nhau giữa các giá trị; các phép chọn min/max và so sánh với cận vì thế giữ nguyên quyết định khi toàn bộ utility được biến đổi đồng nhất.

Kết luận 3 — Kiểm thử và điều hiểu rõ hơn. Log ghi nhận cả 9 test nhóm đều đạt. Qua T1, nhóm hiểu rằng value đúng chưa đủ để xác nhận action đúng. Nhánh B trả 10 với cờ `UPPER` nhưng giá trị thật là 2. Nếu MAX cập nhật action bằng phép so sánh không nghiêm, thuật toán có thể trả $(10, B)$ thay vì $(10, A)$. Vì vậy, test cần kiểm tra cả value, action và ý nghĩa cận; khi chọn action phải giữ phép so sánh nghiêm.

Hai giới hạn của thực nghiệm.

1. Các cây đều là cây nhị phân nhỏ, tối đa 16 lá; kết quả chưa phản ánh đầy đủ hiệu quả trên cây sâu hoặc có hệ số phân nhánh lớn.
2. Nhóm chưa thực hiện đo thời gian lặp và chưa khảo sát nhiều seed cho N1/N2; mức tiết kiệm lá quan sát được chưa đủ để khái quát mức tăng tốc.

Một cải tiến có thể khảo sát tiếp. Bổ sung sắp xếp thứ tự xét con theo ước lượng: MAX ưu tiên nhánh có giá trị dự đoán cao, MIN ưu tiên nhánh thấp. So sánh số lá đã đọc với LR/RL cố định, đồng thời kiểm tra giá trị gốc và tính tối ưu của action; khi có nhiều hành động đồng hạng cần nêu rõ quy tắc xử lý hòa.

10. Kiểm tra trước khi nộp

Hạng mục	Kết quả / người kiểm
Báo cáo, bài tay và 6–8 slide	[Điền]
24+8 cấu hình / trace T0	[Điền]
9 test riêng, mỗi người ít nhất 2 / log thật	[Điền]
README_NOP, dữ liệu và đủ Python	[Điền]
ZIP giải nén chạy lại trên máy khác	[Điền]
Bốn người đã tập bảo vệ 15–20 phút	[Điền]

Tên ZIP: [Điền].

Mốc ngày 5 / ngày 10 và tệp minh chứng: [Điền].

Bốn thành viên xác nhận bài nộp và nội dung phụ trách: [Điền].